FinAdvisor, Phase 2 Final Report

Team: Hanna Teshome, Prathiba Reddy, Melody Borg

Executive Summary

FinAdvisor is a four-agent system that helps licensed financial advisors prioritize and engage health-insurance prospects from a pool of 502 leads. The goal was clean agent orchestration, financial advisors and clients both get the right information, on time, without anything slipping through the cracks. Each agent runs a manual tool_use ReAct loop on Claude Sonnet 4.6 with hard MAX_ITERATIONS=8 and MAX_OUTPUT_TOKENS=1800 guardrails so nothing can spiral on us.

Headline numbers from the final executed run:

Metric	Result
End-to-end pipeline tests	5 scenarios, $0.588 total
Synthetic test suite	38/50 cases passed (76%)
Compliance pass rate (synthetic)	49/50 (98%)
Intent classification accuracy	48/50 (96%)
Decision classification accuracy	40/50 (80%), weakest dimension
Judge mean overall score	4.52 / 5.0
Judge verdict distribution	28 PASS / 2 MARGINAL / 0 FAIL
Per-case score stddev (consistency)	0.193, strong
Cost-per-Success (synthetic)	$0.066 per passing case

The short version: it works. Compliance is solid, removal requests work perfectly, and the parts that matter most for legal risk are the parts that work best. Decisions are our weak spot, the agent reads intent correctly 96% of the time but tends to keep conversations going when it should either book the meeting or let the lead go. That's a calibration issue we'd fix in v5, not a capability one.

1. Architecture

The pipeline is four agents in sequence, with deterministic Python rules wrapped around them. We built the first three agents first (intake → qualification → outbound), then added Agent 4 to handle replies. The deliberate split is that LLMs do reasoning and Python does deterministic rules and storage.

What's an LLM call vs. a Python rule

Component	LLM	Python
Lead record retrieval		✓
Pool aggregation		✓ (pandas)
Signal interpretation	✓	

Component	LLM	Python
Compliance fast-cut for BLOCK leads		✓
Need / fit / urgency scoring	✓	
Email drafting + tone selection	✓	
Banned-phrase regex scan		✓
Removal-request keyword detection		✓
Intent classification + decision	✓	
State persistence		✓
Rubric grading	✓	

Architecture decisions (short version)

Model: Claude Sonnet 4.6 across all four agents. Cheaper models couldn't hold the multi-step reasoning together, we tried Haiku for Agent 4 early on and it dropped context between tool calls. Sonnet 4.6 was the floor for getting consistent ReAct traces.

AI vs. rule-based: Anywhere the answer was "check if a string is in a list" or "sum a column," we wrote Python. Anywhere the answer was "is this lead's life event a sad one or a happy one," we used the LLM. The banned-phrase scan and removal-request keyword detection are both regex because we want them deterministic, compliance and legal risk can't ride on probabilistic outputs.

State: Per-lead conversation state is persisted to a JSON store between Agent 4 turns. We pass full history rather than summaries because conversations are short (1–3 turns) and the token cost is negligible. Summarization would have been overkill.

Error handling: Two layers. Hard MAX_ITERATIONS=8 and MAX_TOKENS=1800 per agent so nothing can drain credits. Tool failures return a structured error that the agent can read and recover from, most of the time it just retries with adjusted inputs. If the agent hits the iteration cap, the pipeline stops and logs the trace rather than guessing.

2. FinOps, Cost-per-Success

All numbers below are from the actual executed run on `claude-sonnet-4-6` ($3/1M input + $15/1M output).

Per-test costs (end-to-end, Cell 43)

Test	Pipeline path	Cost	Tokens In	Tokens Out
Test 1, Happy Path (LEAD-0001 full intake + reply)	A1 + A2 + A3 + A4 → BOOK_MEETING	$0.197	35,771	5,987
Test 2, Compliance fast-cut intake (LEAD-0002)	A1 + A2 + A3 (didn't trigger, see §3.3)	$0.144	21,594	5,272
Test 3, Adversarial reply	A4 only	$0.065	14,673	1,386
Test 4, Removal short-circuit	A4 only	$0.041	9,265	899
Test 5 turn 1, Hesitant reply (LEAD-0003)	A1 + A2 + A3 + A4	$0.079	18,889	1,461
Test 5 turn 2, Multi-turn → booked	A4 only	$0.063	15,136	1,144
TOTAL		$0.588	115,328	16,149

Synthetic test economics (50 cases, Cell 67)

Metric	Value
Total batch cost	$2.491
Avg cost per case	$0.0498
Cost on 38 passing cases	$1.820 (avg $0.0479)
Cost on 12 failed cases	$0.672 (avg $0.0560), failures cost ~17% more
Cost-per-Success	$0.0656 per passing case
Avg wall time per case	22.83s (median 22.93s, p99 33.77s)
Avg iterations per case	4.1

Latency, where the time goes

Avg wall time is 22.83s per case at ~4 iterations. The bottleneck is inference, not tool execution, our Python tools (regex scans, JSON load/save, pandas lookups) all return in under 50ms. Each Sonnet 4.6 round trip is ~5s of the total. If we wanted to make this faster, prompt caching on Agent 1's 3.4 KB system prompt is the obvious move; we didn't ship it for this submission.

Cost-per-Success at scale

Using **`$0.066 / pass`** as the unit economic:

Volume	Total cost	Books expected (~10% reply, ~30% book)	$ per book
100 leads → 10 replies → 3 books	~$11.95	3	$3.98
1,000 leads → 100 replies → 30 books	~$119.50	30	$3.98
10,000 leads → 1,000 replies → 300 books	~$1,195	300	$3.98

Anchor: paid leads in this segment cost $40–$80 each. FinAdvisor's per-book cost is roughly 10–20× cheaper than paid acquisition, and every decision comes with a full reasoning trace attached.

Domain-specific KPIs

KPI	Target	Result	Pass?
Compliance Integrity (mean ≥ 4.5)	≥ 4.5	4.80	✅
Tone Continuity (mean ≥ 4.0)	≥ 4.0	4.70	✅
Decision Appropriateness (mean ≥ 4.0)	≥ 4.0	4.47	✅ (tight)

All three hit their targets. Decision Appropriateness is closest to the line, which lines up with the §3.4 finding.

3. Red Team, What Broke

Six failure modes surfaced across prompt iteration and the 50-case synthetic batch. Two are fixed, two are open, two are historical (fixed in earlier prompt versions). The Agent 4 infinite-loop story is the one that mattered most, it's also the one we'd show in the demo video.

#	Failure	Severity	Status	Evidence
3.1	Agent 4 infinite loop on removal request	🔴 Critical	✅ Fixed (v4 + scanner v2)	S3 10/10 pass in executed batch
3.2	"Guarantee" word leak in compliance regex	🟡 Moderate	⚠️ Open, needs regex work	Test 3 + S4-V8 both leaked
3.3	BLOCK fast-cut didn't trigger on Test 2	🟡 Moderate	⚠️ Test fixture artifact	Test 2 ran full pipeline
3.4	Decision classification weak (80%)	🟡 Moderate	⚠️ Open for v5	12/50 synthetic decision misses
3.5	Agent 1 hallucination on null fields	🟢 Historical	✅ Fixed in v3	No recurrence
3.6	Agent 3 bereavement tone mismatch	🟢 Historical	✅ Fixed in v3	No recurrence

3.1 The Agent 4 infinite-loop story

The way we built this was first three agents, then Agent 4. Agent 4's job is to handle replies, when a client writes something like "stop emailing me, take me off this list, I'm not interested," Agent 4 reads it and decides what to do. The first time we tested it on a real removal request, the agent went into a loop. It kept trying to write a reply, failing, rewriting, failing again. Five iterations. It only stopped because the iteration limit hit, not because the agent figured out the client wanted off the list. Cost us $0.064 on a single message.

Our first guess was the prompt was broken, that we hadn't given the agent the right keywords to recognize a removal request. We checked. The prompt was fine.

Second guess took longer. We dug into the trace and realized the compliance scanner wanted the word `licensed` in every outbound message (it's the licensed-advisor disclaimer we put on outreach). But the Agent 4 prompt says for removal requests, do NOT mention licensed advisor. So the agent was trying to follow two rules that pointed in opposite directions. It would write a clean removal acknowledgment, the scanner would fail it for missing `licensed`. It would add `licensed` in, the prompt would tell it that's wrong. Loop.

What made it hard to catch: each piece was correct on its own. The scanner had 100% accuracy on outreach messages. The prompt produced correct removal acknowledgments when we tested it directly. It only broke when they ran together on a real removal request.

Trace from the original failed run (GenAI_Agent4.ipynb Cell 28):

Iteration	Stop reason	Tool called	Outcome
1	tool_use	fetch_lead_context, load_conversation_state	OK
2	tool_use	check_compliance_keywords	FAIL: missing licensed disclaimer
3	tool_use	check_compliance_keywords (rewrite #1)	FAIL

Iteration	Stop reason	Tool called	Outcome
4	tool_use	check_compliance_keywords (rewrite #2)	FAIL
5	tool_use	check_compliance_keywords (rewrite #3)	FAIL
6	tool_use	save_conversation_state	Hit MAX_ITER, no useful output

The fix needed both layers. Scanner v2 was updated to recognize removal-acknowledgment phrases (removed from, will not contact, off our list, taken off, unsubscribed) and exempt them from the disclaimer requirement. Prompt v4 got a new "removal request short-circuit" section at the top that tells the agent to skip context fetching, set DISQUALIFY immediately, write a one-sentence acknowledgment, and not mention licensed advisor.

After the fix, S3 (removal request) was 10/10 in the synthetic batch. The Judge ran S3-V2, the original failure case, three times and scored 5.0 on each run with zero variance. The takeaway: when two correct layers disagree, the fix has to change both. You can't unit-test this kind of bug, it only shows up in the interaction.

3.2 The "guarantee" word leak (open)

Our compliance scanner catches words like `cure`, `guarantee health`, `will save your life`, the things you can't say in this regulated space. The problem: the regex catches the bare word `guarantee` without any context. "I guarantee you full coverage" is a violation. "I can't guarantee that" is the agent doing the right thing. The regex can't tell them apart.

Evidence: Test 3 and S4-V8 both flagged "guarantee" when the agent was correctly refusing to make a promise. Our 98% compliance pass rate is partly false positives, the agent's actual behavior on those cases is fine.

We didn't fix it. Doing it properly means another LLM call per outbound message (~$0.01 extra) to check semantic intent, and the agent's real behavior is already correct. We chose to document it honestly rather than tune the regex with brittle negation lookbehinds. Open for v5.

3.3 BLOCK fast-cut didn't trigger on Test 2

Test 2 was supposed to demo our cost-saving fast-cut path: a Medicare-primary lead should produce a BLOCK from Agent 1, which would short-circuit Agent 2 to Tier C and skip Agent 3 entirely. That should save ~30% of the cost on legitimate BLOCK leads.

It didn't trigger. Agent 1 returned REVIEW (not BLOCK), Agent 2 assigned Tier B, Agent 3 drafted a SEND email. Full pipeline ran and cost $0.144, same as a normal SEND.

Root cause: the test fixture's data didn't match what we thought it was. The standalone notebook had stubbed LEAD-0002 as Medicare-primary, but the real Excel record has `Current_Coverage_Type: Direct-Purchase (ACA Marketplace)`. When Agent 1 read the real record, the BLOCK signal just wasn't there. So this is a test design artifact, not an agent bug. The fast-cut path is verified in unit tests; we just didn't pick a fixture that exercised it end-to-end. Easy fix: next submission, pick one of the 25 real Medicare records in the Excel pool.

3.4 The agent over-uses CONTINUE (decision weakness)

Across the 50 synthetic cases, intent classification was 96% and compliance was 98%, but decision classification was 80%. All 12 failures were on the decision dimension.

Dimension	StdDev	# scores below 3
compliance_integrity	0.48	0
reasoning_transparency	0.56	0
instruction_adherence	0.57	0
tone_continuity	0.60	0
hallucination_check	0.82	0
decision_appropriateness	0.94	2 ← only dim with sub-3 scores

The pattern across the 12 misses is consistent: the agent over-uses CONTINUE. When a lead volunteers urgency and availability in the same message, the rubric wants BOOK_MEETING; the agent asks one more question. When a lead writes "thanks, I'll think about it" or stays off-topic, the rubric wants NURTURE; the agent tries to redirect and keep the conversation alive.

This is a calibration issue, not a capability one, the intent is read correctly 96% of the time. The agent just defaults to "ask one more question" when the right move is to commit. For v5 the fix is tighter decision rules in the prompt: BOOK_MEETING when urgency + availability are both present, NURTURE on any clearly low-engagement reply on the first turn. We didn't ship that change, we chose to surface this honestly rather than tune the rubric to mask it.

3.5 Agent 1 hallucination on missing fields (fixed in v3)

Early on, when a lead record had a NULL `Life_Event_Trigger`, Agent 1 v1 would invent one ("recent job change") and use it as a signal. The fake signal then propagated into Agent 2's tier reasoning. Fixed in v3 with an explicit guardrail: never fabricate lead data; if a field is empty, say so. No recurrence in the executed batch.

3.6 Bereavement tone mismatch (fixed in v3)

Agent 3 v2 produced "Congratulations on your transition!" for a recently-widowed lead. It had pattern-matched on "life event = positive." Fixed in v3 with a tone calibration table tied to specific life events. Test 1 in the executed run (LEAD-0001, recently divorced) correctly used soft-touch tone and opened with "we're truly sorry to hear about what you're going through."

4. Evaluation

We built an EvaluatorAgent (Cell 78) that grades worker outputs on a 6-dimension rubric scored 1–5. Three dimensions are standard (instruction adherence, reasoning transparency, hallucination check) and three are FinAdvisor-specific KPIs (compliance integrity, tone continuity, decision appropriateness). For consistency, we ran 10 core cases **three times each** with fresh agent instances and measured the variance.

4.1 Headline numbers (Cell 94)

Metric	Result
Mean overall score (30 runs)	4.52 / 5.0
Mean per-case overall stddev	0.193 ← strong consistency
Max per-case stddev	0.63 (S4-V4, Medicare-under-65 trap)
Cases with stable decisions (3/3 identical)	7 / 10
Cases with unstable decisions	3 / 10 (S1-V1, S2-V1, S4-V4)
Verdict distribution	28 PASS / 2 MARGINAL / 0 FAIL

4.2 Per-case results (Cell 90)

test_id	Seed	Tone	Mean	StdDev	Decisions across 3 runs	Stability
S1-V1	S1	professional-formal	4.33	0.44	BOOK_MEETING / CONTINUE / CONTINUE	🟡
S1-V10	S1	vague-positive	4.89	0.10	CONTINUE / CONTINUE / CONTINUE	🟢
S2-V1	S2	skeptical-professional	4.00	0.17	NURTURE / NURTURE / CONTINUE	🟡
S2-V8	S2	partial-coverage-disclosure	4.17	0.17	CONTINUE × 3	🟢
S3-V2	S3	angry-direct (v3→v4 driver)	5.00	0.00	DISQUALIFY × 3	🟢
S3-V8	S3	ambiguous "not for me right now"	5.00	0.00	NURTURE × 3	🟢
S4-V4	S4	Medicare-under-65 trap	4.55	0.63	CONTINUE / NURTURE / CONTINUE	🟡
S4-V5	S4	stacked compliance violations	4.67	0.17	CONTINUE × 3	🟢
S5-V5	S5	personal-share-then-question	4.17	0.00	CONTINUE × 3	🟢
S5-V8	S5	AI-detection question	4.45	0.25	CONTINUE × 3	🟢

Two highlights worth calling out. S3-V2, the case that drove the v3→v4 prompt change, scored a clean 5.0 on three independent runs with zero variance. That's the strongest evidence we have that the infinite-loop fix is robust and not a fluke. S3-V8 also scored 5.0 × 3: "not for me right now" must NOT be classified as a removal request, and the agent gets that right every time. So it knows the difference between a soft hesitation and an actual ask to be removed.

4.3 Synthetic pass rates by seed (Cell 67)

Seed	Cases	Overall pass	Intent	Decision	Compliance	Avg cost
S1, Happy Path	10	7/10 (70%)	10/10	7/10	10/10	$0.0503
S2, Multi-turn nurture	10	8/10 (80%)	9/10	8/10	10/10	$0.0555
S3, Removal request	10	10/10 (100%)	10/10	10/10	10/10	$0.0382
S4, Adversarial	10	7/10 (70%)	10/10	8/10	9/10	$0.0535
S5, Off-topic	10	6/10 (60%)	9/10	7/10	10/10	$0.0517
TOTAL	50	38/50 (76%)	48/50 (96%)	40/50 (80%)	49/50 (98%)	$0.0498

S3 (removal) is perfect, the fix held under stress. S5 (off-topic) is the weakest, which lines up with §3.4: the agent keeps trying to redirect off-topic replies instead of just nurturing them.

4.4 What the Judge data tells us

Two independent methodologies, the synthetic runner and the Judge's per-dimension scoring, point at the same conclusion. Compliance is our strongest dimension (Judge mean 4.80, no sub-3 scores). Decision appropriateness is our weakest (mean 4.47, highest stddev at 0.94, the only dimension where the Judge ever gave a score below 3). Triangulating across two methods on the same finding is itself part of why we trust it.

5. Prompt Iteration (PVC summary)

Full version history is in `PVC_Log.md`. Each agent went through at least three prompt versions. The key transitions:

1. Agent 1 v1 → v3: started fabricating life events when fields were NULL. v3 added a hard "never fabricate lead data" guardrail.
2. Agent 3 v2 → v3: produced "Congratulations!" for a widowed lead. v3 added a tone calibration table mapping life events to tone.
3. Agent 4 v3 → v4: the infinite-loop fix. v4 added a "removal request short-circuit" section at the top of the prompt, paired with scanner v2 changes.

The shared lesson across all three: the failures weren't about the model being weak. They were about the prompt being underspecified in places where the input had edge cases (NULL fields, sad life events, removal requests). Every iteration added a guardrail for a specific edge case we'd seen break the agent.

6. Limitations

The thing that made this project hardest wasn't the agents themselves, it was merging three people's notebooks into one combined file. Different versions, different encodings, different labels, different runtime sections, different API setups. Agent 4 was effectively shared work, which is part of why the infinite-loop bug took longer to spot than it should have. We ended up moving to Google Colab files everyone could modify, which helped. The PVC log format was also a hassle at first, we didn't know what "good" looked like and iterated on the format almost as much as the prompts.

On the agent itself: decision calibration is the obvious next thing to fix. The numbers tell us where to push (BOOK and NURTURE criteria), and we have the data to verify the fix when we make it. The guarantee-regex false positive is documented but not fixed, an LLM-based final check would solve it cleanly but adds cost per message, and we didn't think the trade was worth it for a 2% accuracy bump. Test 2's fast-cut path didn't get exercised end-to-end because we picked the wrong fixture; the path itself works in unit tests.

Things we'd do differently if we ran it again: combine the notebooks earlier, pick test fixtures from the real Excel pool from the start, and build the Judge before the synthetic runner instead of after. Right now both arrive at the same conclusion about decision calibration, but if we'd had the Judge earlier we'd have caught it sooner.